Multi-Agent Systems (MAS) - Assignment 1

Name: Nevil C Prince
Roll Number: EP23BTECH11019

August 21, 2026

Problem 1

Problem Statement

Simulate the following tasks and give your inferences and all the associated codes.

1. For $N = 5$, generate and plot a random undirected graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$ using Erdos-Renyi model.
   (a) Report the adjacency matrix, degree matrix, incidence matrix, and Laplacian matrix
   (b) Compute and report the eigenvalues of Laplacian matrix. Report the algebraic connectivity λ_2 value. Explain the properties of the graph using the computed λ_2 quantity and mention why it is so important.
   (c) Argue why high connectivity (i.e., each nodes having more connections) is a double-edged sword for agents in a network.
2. Assume the number of agents $N = 10^3$ and generate a random undirected connected graph (having spanning tree) $\mathcal{G}(\mathcal{V}, \mathcal{E})$ with N agents using Erdos-Renyi model and plot it.
   (a) Highlight the node(s) with the highest number of neighbours with bright colour. Compute the mean degree, mean edges from the generated random graph and compare that with the theoretical value to see if it matches. Plot the diameter of the graph by colouring the corresponding edges.
   (b) Pick any two nodes in random from the generated graph to compute the shortest path between them using any Shortest Path (SP) algorithm like Dijkstra algorithm and plot the shortest path between them in the graph.
   Note: Both Matlab & Python have readily available package for plug and use, so that you need not write the code for the SP algorithm.

Solution

Part 1: $N = 5$ Erdos-Renyi Graph

(a) Graph Matrices:
The numerical arrays representing the Adjacency, Degree, Incidence, and Laplacian matrices are generated by the provided Python code at the end of this problem.

(b) Laplacian Eigenvalues and Algebraic Connectivity:
The consensus dynamics depend on the eigenvalues of the Laplacian matrix. The Laplacian matrix L always has a trivial eigenvalue of $\lambda_1 = 0$, with an associated consensus mode eigenvector of $\frac{1}{\sqrt{N}}\mathbf{1}_N$. The second smallest eigenvalue, λ_2, represents the algebraic connectivity. The magnitude of λ_2 is critically important because it dictates the decay rate of the disagreement modes (where eigenvalues are $\mu_i = 1 - \varepsilon\lambda_i$ in discrete time). A higher λ_2 ensures that the network converges to the agreement state much faster.

(c) High Connectivity as a Double-Edged Sword:
High connectivity in a multi-agent system provides both significant benefits and severe vulnerabilities:

- **The Advantage:** High connectivity implies a denser network with more edges, which increases the algebraic connectivity (λ_2) and speeds up convergence to consensus[cite: 3]. Furthermore, a highly connected graph is more likely to satisfy the $(F+1, F+1)$-robustness condition, which is a necessary and sufficient condition for algorithms like the Weighted Mean Sub-sequence Reduced (W-MSR) to achieve resilient consensus under an F-total malicious model.
- **The Disadvantage:** Conversely, high connectivity means malicious nodes can also possess a high degree, allowing their toxic influence to reach a large number of benign agents directly. In resilient consensus strategies like W-MSR, where nodes must filter and ignore extreme values, an adversarial node with many connections can force numerous honest neighbors to simultaneously discard valid data points.

Part 2: $N = 10^3$ Erdos-Renyi Graph

(a) Mean Degree, Mean Edges, and Diameter:
For an Erdos-Renyi graph $\mathcal{G}(N, p)$, the theoretical formulas are defined as:

- Mean degree: $c = (N-1)p$.
- Mean edges: $m = p\binom{N}{2}$.
- Establishing the relationship: $c = \frac{2m}{N}$.

The attached Python code computes the empirical mean degree and edge count from the generated graph and compares them to these theoretical expectations. The diameter of the graph is computed and plotted with orange edges.

(b) Shortest Path:
The shortest path between two randomly chosen nodes is computed using Dijkstra's algorithm and is visualized in the network plot with bright green edges. The node with the highest degree is highlighted in red.

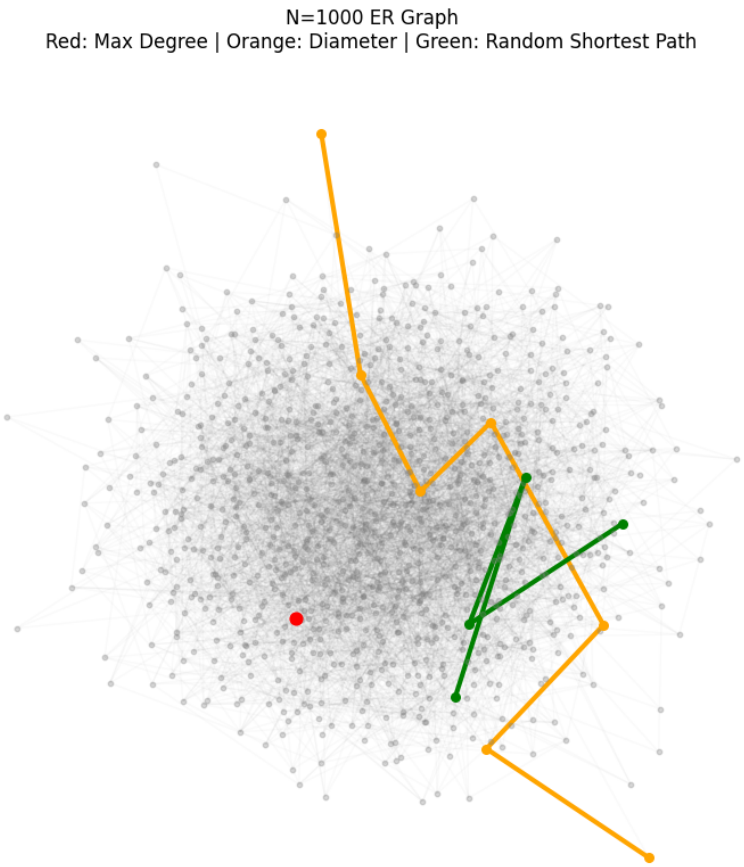


Figure 1: $N = 1000$ Erdos-Renyi graph. Red: highest-degree node. Orange: diameter path. Green: random shortest path.

Python Code for Problem 1

```python
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt
import random

# ========================================
# Part 1: N = 5 Erdos-Renyi Graph
# ========================================
N_small = 5
p_small = 0.5
G_small = nx.erdos_renyi_graph(N_small, p_small, seed=42)

# (a) Matrices
A = nx.adjacency_matrix(G_small).todense()
D = np.diag([d for n, d in G_small.degree()])
Inc = nx.incidence_matrix(G_small).todense()
L = nx.laplacian_matrix(G_small).todense()

print("--- N=5 Graph Matrices ---")
print("Adjacency Matrix:\n", A)
print("Degree Matrix:\n", D)
print("Incidence Matrix:\n", Inc)
print("Laplacian Matrix:\n", L)
```

```
24
25 # (b) Eigenvalues & Algebraic Connectivity
26 eigenvalues = np.linalg.eigvalsh(L)
27 eigenvalues = np.sort(eigenvalues)
28 lambda_2 = eigenvalues[1]
29 print(f"Laplacian Eigenvalues: {eigenvalues}")
30 print(f"Algebraic Connectivity (lambda_2): {lambda_2:.4f}\n")
31
32 # Plotting N=5 Graph
33 plt.figure(figsize=(4, 4))
34 nx.draw(G_small, with_labels=True, node_color='lightblue', font_weight='bold')
35 plt.title("N=5 Erdos-Renyi Graph")
36 plt.show()
37
38 # ==========================================
39 # Part 2: N = 1000 Erdos-Renyi Graph
40 # ==========================================
41 N_large = 1000
42 p_large = 0.01
43 G_large = nx.erdos_renyi_graph(N_large, p_large, seed=42)
44
45 # Ensure graph is connected
46 while not nx.is_connected(G_large):
47     G_large = nx.erdos_renyi_graph(N_large, p_large)
48
49 # (a) Highlight max degree node and compute Means
50 degrees = dict(G_large.degree())
51 max_degree_node = max(degrees, key=degrees.get)
52
53 empirical_mean_deg = sum(degrees.values()) / N_large
54 empirical_mean_edges = G_large.number_of_edges()
55
56 # Theoretical values
57 theoretical_mean_deg = (N_large - 1) * p_large
58 theoretical_mean_edges = p_large * (N_large * (N_large - 1)) / 2
59
60 print("--- N=1000 Graph Properties ---")
61 print(f"Empirical Mean Degree: {empirical_mean_deg} | Theoretical: {
    theoretical_mean_deg}")
62 print(f"Empirical Mean Edges: {empirical_mean_edges} | Theoretical: {
    theoretical_mean_edges}")
63
64 # Compute Diameter Path
65 periphery_nodes = nx.periphery(G_large)
66 source_node = periphery_nodes[0]
67 path_lengths = nx.single_source_shortest_path_length(G_large, source_node)
68 target_node = max(path_lengths, key=path_lengths.get)
69 diameter_path = nx.shortest_path(G_large, source_node, target_node)
70 diameter_edges = list(zip(diameter_path, diameter_path[1:]))
71
72 # (b) Shortest path using Dijkstra between two random nodes
73 nodes = list(G_large.nodes())
74 rand_start, rand_target = random.sample(nodes, 2)
75 dijkstra_path = nx.dijkstra_path(G_large, rand_start, rand_target)
76 dijkstra_edges = list(zip(dijkstra_path, dijkstra_path[1:]))
77
78 # Plotting properties
79 pos = nx.spring_layout(G_large, seed=42)
80 plt.figure(figsize=(10, 10))
81
82 # Draw all nodes and edges (very lightly)
83 nx.draw_networkx_nodes(G_large, pos, node_size=10, node_color='gray', alpha
    =0.3)
```

```
nx.draw_networkx_edges(G_large, pos, alpha=0.02)

# Highlight max degree node (Red)
nx.draw_networkx_nodes(G_large, pos, nodelist=[max_degree_node], node_size=60,
    node_color='red')

# Highlight Diameter (Orange)
nx.draw_networkx_nodes(G_large, pos, nodelist=diameter_path, node_size=30,
    node_color='orange')
nx.draw_networkx_edges(G_large, pos, edgelist=diameter_edges, edge_color='
    orange', width=3)

# Highlight Random Shortest Path (Green)
nx.draw_networkx_nodes(G_large, pos, nodelist=dijkstra_path, node_size=30,
    node_color='green')
nx.draw_networkx_edges(G_large, pos, edgelist=dijkstra_edges, edge_color='green
    ', width=3)

plt.title("N=1000 ER Graph\nRed: Max Degree | Orange: Diameter | Green: Random
    Shortest Path")
plt.axis('off')
plt.show()
```

Problem 2

Problem Statement

We denote the ith component of a vector z at time $k \in \mathbb{N}$ $\overline{(x_k)}_i$. Consider a linear system given by $x_{t+1} = Ax_t$,t=1,..., T, with $x_t \in \mathbb{R}^n$ and $A \in \mathbb{R}^{n \times n}$. Find the matrix $S \in \mathbb{R}^{n \times n}$ that depends upon the matrix A to measure the n numbers $\{(x_1)_1, (x_2)_2, ..., (x_n)_n\}$, i.e., the th component of the state at time i, for $i = 1, ..., n$ such that

$$\begin{bmatrix} (x_1)_1 \\ (x_2)_2 \\ \vdots \\ (x_n)_n \end{bmatrix} = Sx_1.$$

Solution

Given a discrete-time linear system $x_{t+1} = Ax_t$ with $x_t \in \mathbb{R}^n$ and $A \in \mathbb{R}^{n \times n}$. The state vector at any given time k can be evaluated iteratively from the initial state x_1 by repeatedly applying the state update matrix A:

$$x_k = A^{k-1}x_1$$

We need to map the initial state to a specific sequence of components via a linear mapping matrix $S \in \mathbb{R}^{n \times n}$[cite: 4] such that:

$$\begin{bmatrix} (x_1)_1 \\ (x_2)_2 \\ \vdots \\ (x_n)_n \end{bmatrix} = Sx_1$$

Let $e_i \in \mathbb{R}^n$ denote the standard basis vector (a column vector with a 1 in the i-th position and 0 elsewhere). The i-th component of the state vector x_i at time step i can be isolated mathematically using the inner product:

$$(x_i)_i = e_i^\top x_i = e_i^\top (A^{i-1}x_1)$$

Therefore, the entire target measurement column vector can be written by stacking these individual linear mappings:

$$\begin{bmatrix} (x_1)_1 \\ (x_2)_2 \\ \vdots \\ (x_n)_n \end{bmatrix} = \begin{bmatrix} e_1^\top x_1 \\ e_2^\top Ax_1 \\ \vdots \\ e_n^\top A^{n-1}x_1 \end{bmatrix} = \begin{bmatrix} e_1^\top \\ e_2^\top A \\ \vdots \\ e_n^\top A^{n-1} \end{bmatrix} x_1$$

Thus, the linear operator S is directly defined as:

$$S = \begin{bmatrix} e_1^\top \\ e_2^\top A \\ e_3^\top A^2 \\ \vdots \\ e_n^\top A^{n-1} \end{bmatrix}$$

Problem 3

Problem Statement

Let $A = \begin{bmatrix} B & I_n \\ 0_n & B \end{bmatrix}$ with $B \in \mathbb{R}^{n \times n}$. Compute the following.

1. Find e^{At}
2. Let $n = 2$ with $\alpha \in \mathbb{R}$ $\beta > 0$ and assume $B = \begin{bmatrix} 1 & \beta \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \alpha & 1 \\ 0 & \alpha \end{bmatrix} \begin{bmatrix} 1 & -\beta \\ 0 & 1 \end{bmatrix}$ Compute e^{At}

Solution

Part 1

We are given the block matrix:

$$A = \begin{bmatrix} B & I_n \\ 0_n & B \end{bmatrix}$$

We can decompose A into a diagonal block matrix and a nilpotent block matrix:

$$A = \underbrace{\begin{bmatrix} B & 0_n \\ 0_n & B \end{bmatrix}}_{:=D} + \underbrace{\begin{bmatrix} 0_n & I_n \\ 0_n & 0_n \end{bmatrix}}_{:=N}$$

Since D and N clearly commute $\left(DN = ND = \begin{bmatrix} 0_n & B \\ 0_n & 0_n \end{bmatrix}\right)$, we can exploit the property of matrix exponentials: $e^{At} = e^{(D+N)t} = e^{Dt}e^{Nt}$.

First, compute e^{Dt}:

$$e^{Dt} = \begin{bmatrix} e^{Bt} & 0_n \\ 0_n & e^{Bt} \end{bmatrix}$$

Next, compute e^{Nt}. Since $N^2 = 0$, the Taylor series precisely truncates after the first two terms:

$$e^{Nt} = I_{2n} + Nt = \begin{bmatrix} I_n & tI_n \\ 0_n & I_n \end{bmatrix}$$

Multiplying the two results yields e^{At}:

$$e^{At} = \begin{bmatrix} e^{Bt} & 0_n \\ 0_n & e^{Bt} \end{bmatrix} \begin{bmatrix} I_n & tI_n \\ 0_n & I_n \end{bmatrix} = \begin{bmatrix} e^{Bt} & te^{Bt} \\ 0_n & e^{Bt} \end{bmatrix}$$

Part 2

We are given $n = 2$ and the matrix B:

$$B = \begin{bmatrix} 1 & \beta \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \alpha & 1 \\ 0 & \alpha \end{bmatrix} \begin{bmatrix} 1 & -\beta \\ 0 & 1 \end{bmatrix}$$

Let $P = \begin{bmatrix} 1 & \beta \\ 0 & 1 \end{bmatrix}$. Its inverse is exactly $P^{-1} = \begin{bmatrix} 1 & -\beta \\ 0 & 1 \end{bmatrix}$. Let $J = \begin{bmatrix} \alpha & 1 \\ 0 & \alpha \end{bmatrix}$. Thus, B is expressed directly as a similarity transformation: $B = PJP^{-1}$[cite: 4]. Consequently, $e^{Bt} = Pe^{Jt}P^{-1}$.

We compute e^{Jt} by separating the scalar and nilpotent parts: $J = \alpha I_2 + N_2$.

$$e^{Jt} = e^{\alpha t}e^{N_2 t} = e^{\alpha t}\begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix}$$

Now substitute this back to evaluate e^{Bt}:

$$e^{Bt} = \begin{bmatrix} 1 & \beta \\ 0 & 1 \end{bmatrix}\left(e^{\alpha t}\begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix}\right)\begin{bmatrix} 1 & -\beta \\ 0 & 1 \end{bmatrix}$$

$$e^{Bt} = e^{\alpha t}\begin{bmatrix} 1 & t+\beta \\ 0 & 1 \end{bmatrix}\begin{bmatrix} 1 & -\beta \\ 0 & 1 \end{bmatrix} = e^{\alpha t}\begin{bmatrix} 1 & -\beta+t+\beta \\ 0 & 1 \end{bmatrix} = e^{\alpha t}\begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix}$$

Using the block result from Part 1, we can easily write the final exponential matrix e^{At}:

$$e^{At} = \begin{bmatrix} e^{Bt} & te^{Bt} \\ 0_2 & e^{Bt} \end{bmatrix} = e^{\alpha t}\begin{bmatrix} 1 & t & t & t^2 \\ 0 & 1 & 0 & t \\ 0 & 0 & 1 & t \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Problem 4

Problem Statement

Given an interval [a,b] with $a < b$ and a polynomial $p(x) = \sum_{k=0}^{n-1} c_{k+1}x^k$ with coefficients $c \in \mathbb{R}^n$, prove that both integral and derivative of the polynomial $p(x)$ is a linear function of its coefficients. That is, prove that $\exists \alpha \in \mathbb{R}^n$; $\exists \beta \in \mathbb{R}^n$ such that

$$\alpha^\top c = \int_a^b p(x)dx$$

$$\beta^\top c = p'(x) = \frac{dp(x)}{dx}$$

Solution

A linear mapping maps vectors from a domain vector space to a codomain, preserving addition and scalar multiplication. To prove the operators are linear mappings of the coefficient vector c, we must demonstrate they can be represented as an inner product $\alpha^\top c$ and $\beta^\top c$.

Integral:

$$\int_a^b p(x)dx = \int_a^b \left(\sum_{k=0}^{n-1} c_{k+1}x^k\right)dx$$

Since the integral is a linear operator, we can swap the sum and the integral:

$$\int_a^b p(x)dx = \sum_{k=0}^{n-1} c_{k+1}\int_a^b x^k dx = \sum_{k=0}^{n-1} c_{k+1}\left(\frac{b^{k+1}-a^{k+1}}{k+1}\right)$$

This matches the exact structure of a vector dot product $\alpha^\top c$, where $\alpha \in \mathbb{R}^n$ is definitively isolated as:

$$\alpha_{k+1} = \frac{b^{k+1} - a^{k+1}}{k+1}, \quad \text{for } k = 0, 1, \ldots, n-1$$

Derivative:

$$p'(x) = \frac{dp(x)}{dx} = \frac{d}{dx}\left(\sum_{k=0}^{n-1} c_{k+1} x^k\right)$$

Applying the standard power rule to the summation:

$$p'(x) = \sum_{k=1}^{n-1} c_{k+1} k x^{k-1}$$

This can similarly be formulated as an inner product $\beta(x)^\top c$, where the vector $\beta \in \mathbb{R}^n$ relies on the evaluation point x:

$$\beta_1 = 0, \quad \beta_{k+1} = k x^{k-1}, \quad \text{for } k = 1, \ldots, n-1$$

Because both mathematical operations decompose perfectly into fixed scalar multiplications of the elements of the coordinate vector c, both the definite integral and the derivative are linear functions of c.

Problem 5

Problem Statement

Let $\{u_i | i \in \mathbb{N}_{\geq 1}\}$ be a sequence of vectors. Given $x_1 = 0$, let

$$x_t = \frac{1}{t-1} \sum_{k=1}^{t-1} u_k, \quad \forall t \in \mathbb{N}_{\geq 2}.$$

This represents a recursive mean computation of a collection of vectors by accessing them one by one. Find A, B matrices such that $x_{t+1} = Ax_t + Bu_t$ for $t = 1, 2, ...$

Solution

We are analyzing a discrete-time linear algorithm mapping state $x(k)$ to $x(k+1)$. We are given the recursive mean equation:

$$x_t = \frac{1}{t-1} \sum_{k=1}^{t-1} u_k \quad \forall t \geq 2$$

By shifting the index to $t+1$, the next state is given by:

$$x_{t+1} = \frac{1}{t} \sum_{k=1}^{t} u_k$$

We can separate the summation into the sequence of all previous inputs and the single newest incoming input:

$$x_{t+1} = \frac{1}{t} \left(\sum_{k=1}^{t-1} u_k + u_t\right)$$

Notice that from the explicit definition of x_t, we can reformulate the summation term:

$$(t-1)x_t = \sum_{k=1}^{t-1} u_k$$

Substitute this equality directly back into the separated equation:

$$x_{t+1} = \frac{1}{t}\left((t-1)x_t + u_t\right) = \left(\frac{t-1}{t}\right)x_t + \left(\frac{1}{t}\right)u_t$$

Therefore, to properly fit the required linear system format $x_{t+1} = Ax_t + Bu_t$, the matrices A and B must be time-varying scalar multiples of the identity matrix:

$$A_t = \frac{t-1}{t}I, \quad B_t = \frac{1}{t}I$$